Track Stack: Official Project README

Complete repository documentation, deployment guide, and developer manual.

Club Activity Management for Mirai School of Technology (MSOT)

Built with AI

What This Project Demonstrates

* Directing AI Engineering Tools: Translating high-level educational institution specifications into precise system designs, schema constraints, and secure workflows.
* Overview: Product Architecture & Thinking: Defining user roles, access control boundaries, anti-tamper mechanisms, and intuitive real-world UX for both mobile and desktop.
* Verification & Review: Enforcing strict testing discipline, cryptographic validation, zero-trust RLS policies, and regression auditing on AI-generated code.

Overview

Club data in modern educational institutions is historically fractured across ephemeral WhatsApp group chats, ad-hoc Google Forms, and disconnected spreadsheets. Attendance is vulnerable to proxy signing, evaluation feedback is lost, certificates are forged or unverified, and students graduate without an official, tamper-proof record of their co-curricular achievements.

Track Stack solves this by unifying college club administration into a single, high-integrity platform engineered around three foundational pillars:

1. VERIFIED: Every attendance mark, file submission, rubric evaluation, and certificate is backed by cryptographic proof (rotating HMAC tokens, SHA-256 signatures, and immutable audit logging).

2. LOW EFFORT FOR STAKEHOLDERS: Student coordinators can run a live Saturday Club Day from a phone in single taps; campus managers receive automated, mathematically reconciled monthly reports.

3. STUDENT-OWNED RECORD: Every student maintains an indisputable co-curricular transcript and cryptographic certificates verifiable on public university lookup endpoints.

Features by Role

Admin

* System Configuration: Configure global parameters including `max_clubs_per_student` (default: 2), grace periods, and geofence radii.
* Overview: Unified "Needs Attention" Radar: Real-time alerts highlighting inactive clubs (no sessions in X days) and students with zero co-curricular participation.
* Overview: Immutable Audit Log (`/admin/audit`): Tamper-proof ledger logging every attendance correction (with mandatory reason), score publication, certificate issuance, and role assignment.
* Overview: Lumi AI Console (`/admin/lumi`): Manage knowledge-base FAQs, review downvoted answers, inspect query analytics, and configure assistant guardrails.

Campus Manager

* Campus-Scoped Oversight: Dedicated dashboards monitoring participation rates, attendance trends, and submission health across campus clubs.
* Duty Leave Approval Queue: Review and digitally approve/reject inter-campus Duty Leave requests with comments, generating signed PDF leave letters.
* Automated Monthly Club Reports: Receive end-of-month club performance summaries strictly reconciled against raw database records.

Faculty Advisor

* Club Day Activity Approval: Review, accept, or request revisions on proposed Saturday Club Day session plans submitted by coordinators.
* Rubric Evaluations: Grade student deliverables against 0-10 rubrics (*Content Mastery 40%*, *Presentation 30%*, *Participation 30%*) with draft score concealment until official publication.
* Overview: Achievement Certificate Issuance: Issue verified digital achievement certificates to session winners and top presenters.

Student Coordinator (Mobile-First Run Deck)

* Overview: One-Handed Run Screen (`/activities/[id]/run`): Sticky mobile controls to project rotating QR codes, toggle 1-tap roster statuses (`Present`, `Late`, `Absent`, `Excused`), and award spot recognitions.
* Anti-Proxy QR Projector: Live 25-second countdown timer projecting dynamic QR tokens signed with HMAC-SHA256 and backup 6-digit OTPs.
* Mandatory Correction Reasons: Correcting attendance requires a documented justification recorded in the immutable audit trail.
* Announcements: Post club-wide announcements with optional banner pinning and automated email notifications.

Student

* Overview: QR Check-in Scanner (`/checkin`): Fast in-app camera scanner simulator and manual 6-digit code entry with instant geofence and token verification.
* Overview: Submissions Engine (`/activities/[id]/submissions`): Multi-format deliverable uploads (PDF, PPTX, ZIP, links) with full revision versioning (`v1`, `v2`).
* Overview: Co-Curricular Transcript (`/transcript`): Downloadable, tamper-proof university transcript PDF summarizing club memberships, attendance %, activity scores, and achievements.
* Overview: Event Registrations & Teams (`/events`): Register for campus hackathons and conferences, form teams, invite peers, and monitor waitlists.

Meet Lumi: AI Assistant

Lumi is Track Stack's built-in conversational AI assistant, designed to help students, coordinators, and faculty quickly navigate campus activities and resolve routine questions.

* Dual-Engine Brain: Powered by the Google Gemini SDK (`@google/genai`) using fast Flash models, paired with a robust local knowledge-base fallback engine.
* Multilingual Fluency: Seamlessly understands and responds in English, Hindi, and Hinglish.
* Role & Path Aware: Understands who you are and where you are in the application (e.g., provides scanner guidance when on `/checkin`).
* Read-Only RLS Safe Tools: Can securely look up your registered clubs, attendance percentages, assignment deadlines, verified certificates, and Duty Leave status.
* Pre-Login Help: Available in limited guest mode on the login page to assist prospective students and newcomers.

Clubs Covered

Track Stack is deployed for the Delhi-NCR Campus of Mirai School of Technology, managing 4 core clubs and Kaizen Society's 3 specialized sub-clubs:

Logo	Club	Category	Description	Accent Color
<img src="public/log	NexCell	Entrepreneurship Clu	Startup incubator, p	`#3B9BE0`
<img src="public/log	Malhar	Cultural Club	Music, dance, dramat	`#E8A935`
<img src="public/log	Khel Society	Sports Club	Athletics, inter-col	`#1E6FC0`
<img src="public/log	Kaizen Society	Technical Club	Flagship technical c	`#4FA39A`
<img src="public/log	Kaizen Dev Club (Dev	Software Engineering	Web, mobile, systems	`#4FA39A`
<img src="public/log	CP Club	Algorithms & Problem	Competitive coding,	`#A3E635`
<img src="public/log	Kaizen Intelligence	Artificial Intellige	Machine learning, co	`#D9A441`

Tech Stack

Technology	Purpose	Why We Chose It
Next.js 16 (App Router)	Full-Stack Web Framework	Server Components, fast route hand
TypeScript (Strict)	Static Typing	Absolute type safety across databa
Tailwind CSS	Styling & Design System	Utility-first responsive styling w
Framer Motion	UI Micro-interactions	Subtle state transitions, floating
Recharts	Analytics & Visualizations	Responsive SVG charts for attendan
Supabase (PostgreSQL)	Database & Realtime	Hosted PostgreSQL with strict Row
Zod	Schema Validation	Comprehensive runtime validation f
pdf-lib	PDF Generation	Server-side vector PDF generation
qrcode	QR Generation & Verification	Dynamic SVG and PNG QR encoding fo
Resend	Transactional Emails	High-deliverability notification e
Google Gemini API	AI Assistant (Lumi)	Fast response generation with func
Sharp	Image Processing Pipeline	High-fidelity asset generation, ci
PWA (Manifest + Service Worker)	Mobile Web App	Installable home-screen app with o

Architecture

COMMAND SCRIPT / CONFIGURATION

```
flowchart TD
    subgraph Client["Client Devices (Mobile & Desktop)"]
        Browser["Next.js App / PWA"]
        Scanner["QR Scanner & Camera Simulator"]
        LumiUI["Lumi Chat Assistant"]
    end

    subgraph Server["Next.js Route Handlers & Server Engine"]
        AuthGuard["Auth & Role Guards"]
        CheckinEngine["Anti-Proxy QR Engine\n(HMAC-SHA256 Token Validator)"]
```

How Check-In Stays Secure

1. Rotating Tokens: The coordinator screen generates a fresh token every 25 seconds signed with HMAC-SHA256(sessionId + timestamp + secret). Expired tokens are rejected instantly.

2. Geofence Guard: Checks student GPS coordinates against campus coordinates using the Haversine formula (300m campus radius).

3. Anti-Proxy Device Fingerprinting: Combines browser characteristics to flag multiple check-ins from the same physical device (device_reuse_flag = true).

4. Idempotency Guarantee: Unique composite database constraints prevent duplicate check-ins for the same student on the same session.

How Certificate Verification Works

1. Every issued certificate stores a digital fingerprint: HMAC-SHA256(certId + studentId + eventName + date + secret).

2. This signature is embedded as a QR code and metadata field on the server-generated PDF.

3. When scanned or verified on /verify/[certificateId], the server recalculates the cryptographic signature against canonical database values. If a PDF is altered or the record is revoked, the system returns INVALID-TAMPERED or REVOKED.

Screenshots & Visual Previews

Screen	Target Route	Status
Hero Landing & Public Verify	`/`	![Landing Screen](docs/screenshots
Coordinator 1-Hand Run Deck	`/activities/[id]/run`	![Run Deck](docs/screenshots/run-d
Student Rotating QR Check-In	`/checkin`	![QR Check-In](docs/screenshots/ch
Public Certificate Verification	`/verify/[id]`	![Verify Certificate](docs/screens
Lumi AI Assistant Drawer	Global Overlay	![Lumi AI](docs/screenshots/lumi.p
Admin & Campus Dashboards	`/dashboard`	![Dashboard](docs/screenshots/dash

Getting Started

Prerequisites

* Node.js: v18.17+ or LTS (v20+ recommended)
* Overview: npm or pnpm
* Overview: Supabase Project (hosted or local CLI)
* Overview: *(Optional)* Google Gemini API Key for Lumi AI assistant live generation

One-Command Setup (Recommended)

Run the automated one-command setup script to verify environment variables, process official club and Lumi logos, seed the Delhi-NCR campus, and run integrity checks:

COMMAND SCRIPT / CONFIGURATION

```
# 1. Clone repository
git clone https://github.com/Ayush-git800/Track-Stack.git
cd Track-Stack

# 2. Automated one-command setup
npm run setup

# 3. Start local development server
npm run dev
```

Docker Compose Setup

Run Track Stack in containerized mode with Docker:

COMMAND SCRIPT / CONFIGURATION

```
# Build and run containers in background
docker compose up -d

# View logs
docker compose logs -f
```

Manual Installation & Setup

COMMAND SCRIPT / CONFIGURATION

```
# 1. Install dependencies
npm install

# 2. Configure environment variables
cp .env.example .env.local

# 3. Process logos and seed data
npm run process-logos
npm run seed
```

Environment Variables

Database Setup & Seeding

Variable	Description	Required?	Source / Default
`NEXT_PUBLIC_SUPABASE_URL	Supabase project API URL	Yes	Supabase Project Settings
`NEXT_PUBLIC_SUPABASE_ANO	Supabase public anon key	Yes	Supabase Project Settings
`SUPABASE_SERVICE_ROLE_KE	Privileged service key fo	Optional	Supabase Project Settings
`ADMIN_EMAIL`	Default system administra	Yes	`admin@msot.edu`
`NEXT_PUBLIC_DEMO_MODE`	Enables quick demo login	Yes	`true`
`NEXT_PUBLIC_APP_URL`	Public site domain	Yes	`http://localhost:3000`
`HMAC_SECRET`	Secret key for certificat	Yes	Strong random 32+ char st
`CRON_SECRET`	Secret bearer token for s	Yes	Strong random string
`RESEND_API_KEY`	Resend API key for transa	Optional	Resend Dashboard (console
`GEMINI_API_KEY`	Google Gemini API key for	Optional	Google AI Studio (fallbac
`LUMI_MODEL`	Gemini LLM model identifi	Optional	`gemini-2.5-flash`

COMMAND SCRIPT / CONFIGURATION

```
# Process raw logo assets into circular WebP and PNG files
npm run process-logos

# Apply database migrations to Supabase (via SQL editor or CLI)
# 1. supabase/migrations/001_initial_schema.sql
# 2. supabase/migrations/002_phase2_schema.sql
# 3. supabase/migrations/003_phase3_schema.sql
```

Run Development Server

COMMAND SCRIPT / CONFIGURATION

```
npm run dev
```

Open http://localhost:3000 in your browser.

Demo Accounts

When NEXT_PUBLIC_DEMO_MODE=true, 1-tap demo credentials are provided on the login page:

Role	Name	Email	Permissions
System Admin	Ayush Chauhan	`admin@msot.edu`	Global system control, au
Campus Manager	Sundaram Mishra	`manager.delhi@msot.edu`	Delhi-NCR campus oversigh
Faculty Advisor	Gaurav Kumar	`advisor.kaizen@msot.edu`	Kaizen Society activity a
Student Coordinator	Rishav Raj	`coordinator.kaizen@msot.	Kaizen Dev Club check-in
Student	Chintu	`student.alex@msot.edu`	Member of Kaizen Dev Club

Project Structure

COMMAND SCRIPT / CONFIGURATION

```
Track-Stack/
    public/                # Static assets, PWA manifest, and processed logos
        icons/             # PWA and Lumi square app icons
        logos/             # 512x512 circular PNG & WebP club and Lumi logos
    src/
        app/               # Next.js App Router pages and API routes
            (app)/         # Authenticated application shell (sidebar + header)
                activities/ # Activity planning, run deck, evaluation, submissions
                admin/     # Audit logs, users, settings, and Lumi console
                checkin/   # Student QR scanner and check-in portal
```

Testing & Quality Assurance

Track Stack features comprehensive automated test suites across every functional domain:

COMMAND SCRIPT / CONFIGURATION

```
# Run comprehensive automated test report suite (51/51 tests across all phases)
npm run test:report

# Run high-concurrency load testing benchmark (100, 300, 500 VUs)
npm run test:load

# Run individual test suites
npm test              # Core Phase 1-5 suites
npm run test:verify   # Phase 1 & 2: Database schemas, RLS rules, 2-club limit
npm run test:phase3   # Phase 3: Rotating QR, anti-proxy, geofencing, rubrics
```

Detailed test logs and architecture documentation:

* Overview: Comprehensive Automated Test Report: docs/TEST_REPORT.md
* High-Concurrency Load Testing: docs/LOAD_TEST.md
* Overview: 5-Minute Evaluation Demo Script: docs/DEMO_SCRIPT.md
* Overview: System Architecture & Technical Design: docs/DESIGN.md

Deployment

Vercel (Frontend & Serverless Handlers)

1. Push your repository to GitHub.
2. Import the project into your Vercel Dashboard.
3. Configure the environment variables from your .env.local.
4. Deploy with standard Next.js defaults.

Supabase (Database & Storage)

1. Link your Supabase CLI to your remote project: npx supabase link --project-ref your-ref.
2. Push migrations: npx supabase db push.
3. Set up the storage buckets for activity-submissions and duty-leave-letters.

Roadmap & Known Limitations

* Current Limitations:
* Overview: In local development mode without a Supabase cloud instance, state is managed via an in-memory/mock store fallback.
* Overview: Camera scanning simulator is used when web camera hardware permissions are unavailable.
* Planned Enhancements:
* Overview: Native mobile biometric check-in support.
* Overview: Placement Readiness integration via the authorized `/api/transcript/[studentId]` JSON API.
* Overview: Offline Bluetooth Low Energy (BLE) beacon check-in for lecture halls.

Credits & Acknowledgements

* Overview: Mirai School of Technology (MSOT) for providing the real-world problem brief and institution requirements.
* Overview: Club Leadership & Societies (NexCell, Malhar, Khel Society, Kaizen Society) for their respective brand logos and identity assets.
* Core Open Source & Cloud Tools: [Next.js](https://nextjs.org), [Supabase](https://supabase.com), [Tailwind CSS](https://tailwindcss.com), [Google Gemini](https://ai.google.dev), [Vercel](https://vercel.com), and [pdf-lib](https://pdf-lib.js.org).

Team: Version Controllers

This project was developed by team Version Controllers (Mirai School of Technology):

* Overview: Ayush Chauhan (Team Leader) - [GitHub Profile](https://github.com/Ayush-git800)
* Overview: Rishav Ranjan - [GitHub Profile](https://github.com/rishavgiri1106-dotcom)

License

This project is licensed under the MIT License - see the [LICENSE](./LICENSE) file for details.

* Repository: https://github.com/Ayush-git800/Track-Stack